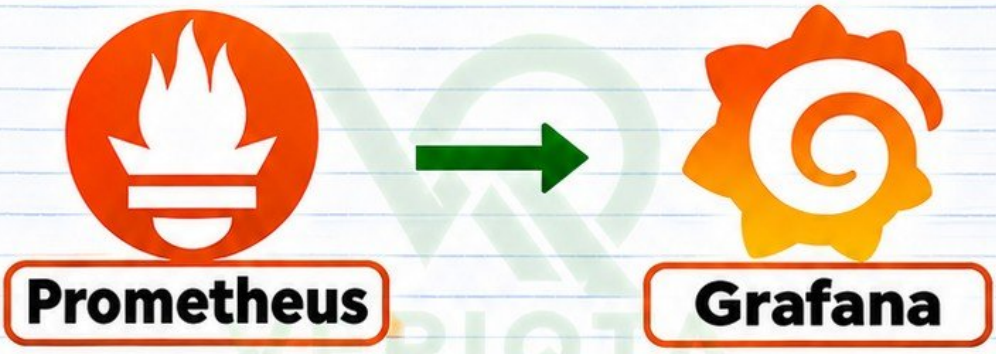
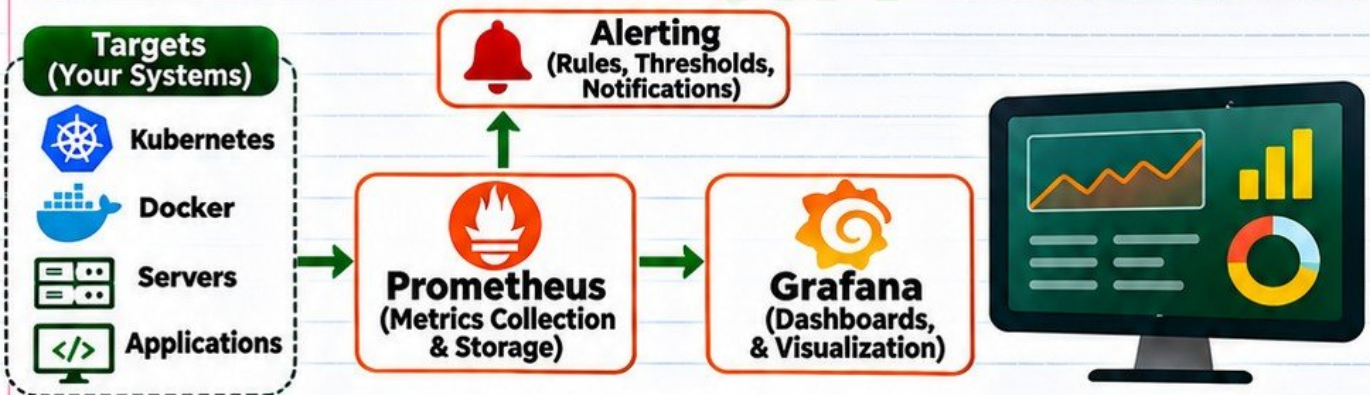
VERIQTA

Prometheus and Grafana Crash Course

Monitoring, Alerting, and Observability for Real-World Systems





What You Will Learn

- Install and configure Prometheus and Grafana.
- Collect metrics from servers, containers, and applications.
- Build useful dashboards for real-time visibility.
- Create alerts and notifications for critical issues.
- Use in a practical, hands-on monitoring project.

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Introduction to Prometheus and Grafana

What is Monitoring?

Monitoring means continuously observing your systems and applications to know their health, performance, and availability.

Why It Matters

- Gives visibility into running systems.
- Helps detect and fix problems early.
- Keeps applications reliable for users.

Key Concepts

Metrics: Numerical data
e.g., CPU, memory, request rate.

Logs: Event records
e.g., errors, warnings, access logs.

Traces: Follow a request across different services to find bottlenecks.

How the Tools Work

Prometheus

- Collects metrics from servers and applications.
- Stores metrics as time-series data.
- Lets you query metrics (with PromQL).

Grafana

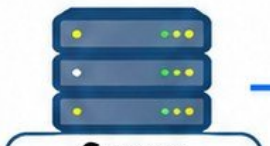
- Visualizes metrics with dashboards.
- Turns raw data into clear graphs and charts.
- Helps you spot trends and problems quickly.

Real-World Example



Detecting a server running out of memory.
Prometheus collects memory usage metrics, and Grafana shows a rising memory graph with an alert when it reaches a critical level.

How It Fits Together



How Prometheus and Grafana Work Together

Monitoring Architecture



A setup where applications expose metrics, Prometheus collects (scrapes) those metrics, and Grafana visualizes them on dashboards.

Metrics Endpoint



A URL (e.g., /metrics) that exposes metrics from an application or server.

Prometheus Pull Model



Prometheus actively pulls (fetches) metrics from targets at regular intervals.

Scraping and Intervals



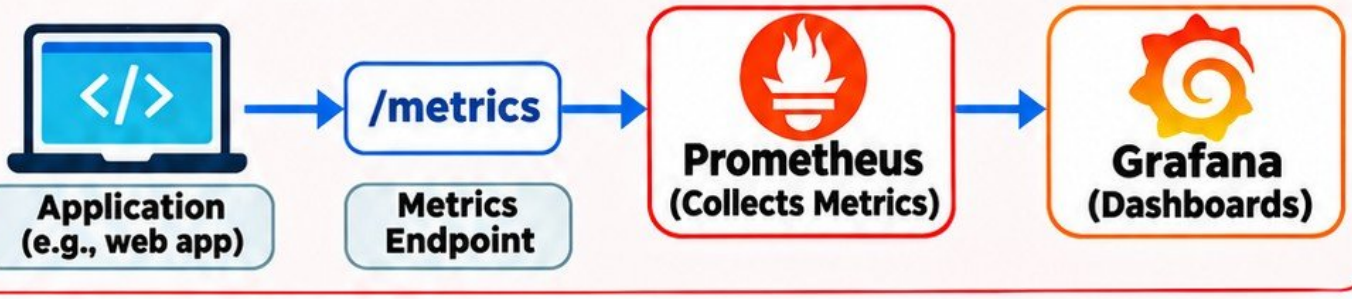
- Scraping = collecting metrics from a target.
- Scrape interval = how often (e.g., every 15s, 30s) Prometheus collects metrics.

How Grafana Queries Prometheus



Grafana does not collect metrics. It sends queries (PromQL) to Prometheus to read the collected data and display it on dashboards.

How It Works (Flow)



Junior Engineer Tip



Grafana does not automatically collect metrics from servers. Prometheus collects the metrics, and Grafana visualizes them.

Understanding Metrics, Labels, and Time Series

What Is a Metric?

A numeric measurement that shows the state or behavior of a system (e.g., CPU usage).

What Is Time-Series Data?

A sequence of metric values collected over time, usually with a timestamp.

Metric Name vs Label

- **Metric name:** What is measured (e.g., cpu_usage_seconds_total).
- **Label:** Extra details about the metric (e.g., instance="server1", env="prod").

Why Labels Matter

Labels add context, help you filter, group, and compare metrics (e.g., by server or environment).

Examples

CPU Usage
(e.g., 75%)

Memory Usage
(e.g., 3.2 GB)

HTTP Requests
(e.g., 200 req/min)

Metric Types

Counter

Always increases (e.g., total HTTP requests).

Gauge

Can go up or down (e.g., CPU usage, memory in use).

Histogram

Measures the distribution of values (e.g., request latency buckets).

Summary

Provides a summary of values (e.g., average or percentiles).

Common Mistake

Creating too many unique label combinations.
This can increase memory usage and slow down Prometheus.

Install Prometheus and Grafana Using Docker Compose

What You Need

Docker
(Run containers)

Docker Compose
(Run multi-container setup)

Setup Steps

1. Check Docker and Compose.
2. Create a project directory.
3. Create a compose.yaml file.
4. Start the monitoring stack.
5. Verify the services are running.
6. Access Prometheus and Grafana.

1 Check Installation

```
$ docker --version
Docker version 27.3.1
$ docker compose version
Docker Compose version v2.29.2
```

Both commands should return a version.

2 Create Project Directory

```
$ mkdir monitoring
$ cd monitoring
```

3 Create compose.yaml

```
services:
  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
```

4 Start the Stack

```
$ docker compose up -d
```

Starts Prometheus and Grafana in the background.

5 Verify Services

```
$ docker compose ps
```

NAME	STATUS	PORTS
prometheus	Up	0.0.0.0:9090→9090
grafana	Up	0.0.0.0:3000→3000

6 Access the Tools (Practice)

Prometheus
http://localhost:9090

Grafana
http://localhost:3000

Practice | Run the setup, open both web interfaces, and confirm they are working.

Understanding the Prometheus Configuration File

What Is prometheus.yml?



The main configuration file for Prometheus. It defines what to scrape, how often, and from where.

Key Configuration Fields

global	Global settings (e.g., scrape interval).
scrape_configs	List of scrape jobs.
job_name	A name for a group of targets.
targets	The endpoints (IP or URL) to scrape.

Example: prometheus.yml

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

What It Means

- **global:** Sets a 15 seconds scrape interval.
- **scrape_configs:** Defines the targets to collect metrics from.
- **job_name:** The name of this scrape job ("prometheus").
- **targets:** The endpoint being scraped ("localhost:9090").

Scrape Interval



How often Prometheus collects metrics (e.g., 15s, 30s, 1m).

Static Targets



A fixed list of endpoints (e.g., localhost:9090, 10.0.0.5:9100).

Job vs Instance



Job: A logical group of targets (e.g., all web servers).



Instance: A single scraped endpoint (e.g., server1:9100).

Practice



Configure Prometheus to scrape its own metrics (using localhost:9090) and verify the target is UP in the Prometheus UI.

Troubleshooting



Invalid YAML Configuration

- Check indentation (use spaces, not tabs).
- Validate YAML syntax.
- Check container logs for the error message (e.g., docker compose logs prometheus).

Understanding Exporters and Metrics Endpoints

What Is an Exporter?

A small program that collects system or application data and exposes it as metrics for Prometheus.

Why Prometheus Needs Exporters

Prometheus collects metrics by scraping a metrics endpoint. Exporters provide that endpoint for servers, databases, and apps.

The /metrics Endpoint

A URL (e.g., /metrics) that returns metrics in a plain text format that Prometheus can scrape.

Common Exporters

- **Node Exporter:** Linux server metrics (CPU, memory, disk, network).
- **cAdvisor:** Container metrics.
- **Nginx Exporter:** Nginx web server metrics.
- **MySQL Exporter:** Database metrics.

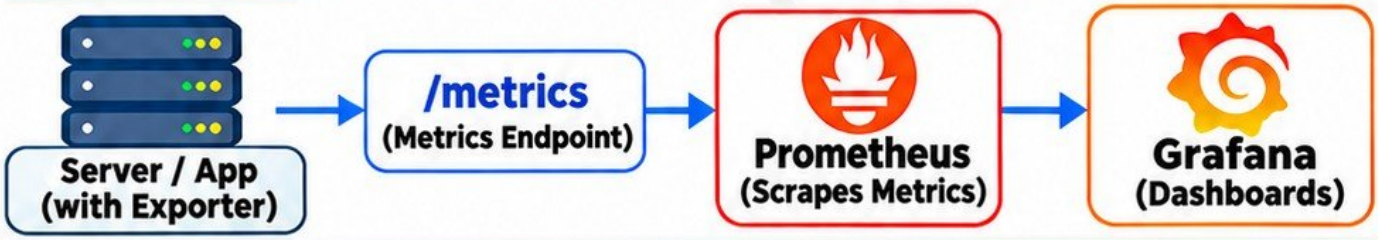
Application Instrumentation

Add a metrics library (e.g., Prometheus client) to your application to expose custom metrics (e.g., request count, latency).

Use curl to Inspect Metrics

```
$ curl http://localhost:9100/metrics
# HELP node_cpu_seconds_total
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
```

How It Works



Practice

- ☑ Use curl to view a /metrics endpoint.
- ☑ Identify metric names and values (e.g., node_memory_MemTotal_bytes).

Security Reminder

Avoid exposing metrics endpoints publicly without appropriate protection (e.g., firewall, authentication, or network restrictions).

Monitor Linux Servers With Node Exporter

What Is Node Exporter?



A Prometheus exporter that exposes Linux server metrics (e.g., CPU, memory, disk, network).

What You Can Monitor

- CPU Usage
- Memory Usage
- Disk Usage
- Network Activity

Deploy Node Exporter (Docker Compose)

```yaml
services:
  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    restart: unless-stopped
```

Add to Prometheus (prometheus.yml)

```yaml
scrape_configs:
 - job_name: "node-exporter"
   static_configs:
    - targets: ["node-exporter:9100"]
```

Host vs Container Metrics

Host metrics: The Linux server itself (CPU, memory, disk, network, etc.).

Container metrics: Resource usage of containers (with additional exporters).

Verify the Target

1. Open Prometheus → Status → Targets.
2. Check that node-exporter is UP (green).

Endpoint	State	Labels
node-exporter:9100	UP	job="node-exporter"

Flow: Linux Monitoring with Node Exporter



Linux Server (Node Exporter) → Prometheus (Scrapes Metrics) → Grafana (Dashboards)

Practice



- Inspect server metrics e.g., CPU, memory, disk usage.
- Run useful queries in Prometheus (e.g., node_cpu_seconds_total, node_memory_MemTotal_bytes).

Troubleshooting



- **Target DOWN:** Check if Node Exporter is running.
- **Connection refused:** Verify the port (9100) and container status.
- Check Docker logs (docker logs node-exporter).

Introduction to PromQL

What Is PromQL?

The query language used in Prometheus to retrieve and analyze metrics.

Vector Types

- **Instant vector:** A single point-in-time result.
- **Range vector:** A range of samples over a time period (e.g., [5m], [1h]).

Metric Selectors and Labels

Select metrics and filter with labels (e.g., http_requests_total {job="web"}).

Query the up Metric

```
up
```

Shows which targets are up (1) or down (0).

rate() and increase()

- **rate(metric[5m]):** Per-second rate over a time range.
- **increase(metric[5m]):** Total increase over the time range.

Aggregations

- **sum():** Add values (e.g., sum(http_requests_total)).
- **avg():** Calculate the average (e.g., avg(cpu_usage)).

Practice: HTTP Request Rate

```
rate(http_requests_total{job="web"}[5m])
```

Returns the per-second HTTP request rate for the web job (over 5 minutes).

Common Mistake

Treating a cumulative counter as a current rate.

Use rate() or increase() for metrics that only go up (e.g., http_requests_total).

Write Useful PromQL Queries

Key Server Monitoring Queries

CPU Utilization

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100
```

Memory Utilization

```
100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
```

Filesystem Usage

```
100 * (1 - (node_filesystem_avail_bytes / node_filesystem_size_bytes))
```

Network Traffic

```
sum by (instance) (rate(node_network_receive_bytes_total[5m]))
```

Important Concepts

- **sum by (...):** Aggregates metrics and groups by a label (e.g., instance).
- **Rate Window:** Choose a suitable window (e.g., [1m], [5m], [15m]) based on data frequency.
- **Label Grouping:** Use labels like instance, job, device to group and compare metrics.

Three Practice Queries

1. **CPU (per server)**

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100
```

2. **Memory (per server)**

```
100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
```

3. **Disk Usage (per mount)**

```
100 * (1 - (node_filesystem_avail_bytes / node_filesystem_size_bytes))
```

How to Choose a Rate Window

Window	When to Use
[1m]	Fast changes (e.g., testing, troubleshooting).
[5m]	General monitoring (recommended).
[15m]	Smoother, long-term trends (less noise).

Practice

- ✅ Build the 3 queries (CPU, memory, disk) in the Prometheus UI.
- ✅ Verify the results return real values for your server (Node Exporter).

Troubleshooting

- **Empty results:** Check the metric name, labels, and if the target is UP.
- **Incorrect label filters:** Use the correct label key and value (e.g., {instance="node-1:9100"}).

Connect Grafana to Prometheus

What Are Grafana Data Sources?

A data source is a connection in Grafana to a system (e.g., Prometheus) that provides data for dashboards and queries.

What You'll Do

- Add Prometheus as a data source.
- Use the correct Prometheus URL.
- Understand Docker service-name networking.
- Test the connection.
- Run a basic PromQL query in Explore.

Step-by-Step: Add Prometheus in Grafana

1. Open Grafana: http://localhost:3000 (default login: admin / admin).
2. Go to "Connections" → "Data sources".
3. Click "Add data source".
4. Select "Prometheus".
5. Set the URL to: **http://prometheus:9090** (Docker service name).
6. Click "Save & test".
7. You should see: **"Data source is working"** (green).

Docker Service-Name Networking

In Docker Compose, containers can reach each other using the service name (e.g., prometheus) instead of localhost.

Example (compose.yaml):

```
services:
  prometheus:
    image: prom/prometheus:latest
  grafana:
    image: grafana/grafana:latest
```

Flow: Grafana to Prometheus

Grafana (Queries & Visualizes) → **http://prometheus:9090** (Docker Network) → **Prometheus** (Provides Metrics)

Run a PromQL Query in Explore

1. In Grafana, go to "Explore".
2. Select Prometheus as the data source.
3. Run a query, e.g.:

```
up
```

4. You should see a time series result (e.g., value 1 for UP targets).

Troubleshooting

Why localhost:9090 May Fail

- Inside the Grafana container, localhost refers to Grafana itself, not Prometheus.
- Use the Docker service name instead (e.g., http://prometheus:9090).
- Also check that Prometheus is running and the URL is correct.

Build Your First Grafana Dashboard

What Is a Grafana Dashboard?

A customizable screen that shows your monitoring data using charts, graphs. and other visualizations.

Key Terms

- **Dashboard:** A page that organizes multiple panels.
- **Panel:** A single visualization (e.g., graph, gauge, stat).
- **Visualization:** A way to display data (e.g., time series, gauge, bar).

Steps: Create Your First Dashboard

1. Open Grafana (http://localhost:3000) and log in.
2. Click "+" → "Dashboard."
3. Click "Add visualization."
4. Select Prometheus as the data source.
5. Add panels for CPU, memory, and disk usage (see queries on the right).
6. Choose a suitable visualization (e.g., Time series or Gauge).
7. Set units (e.g., %, GiB).
8. Click "Save dashboard", give it a name (e.g., Linux Server Health), and save.

Example Panel Queries (PromQL)

CPU Usage

```
100 - (avg by (instance)
(rate(node_cpu_seconds_total
{mode="idle"}[5m])) * 100
```

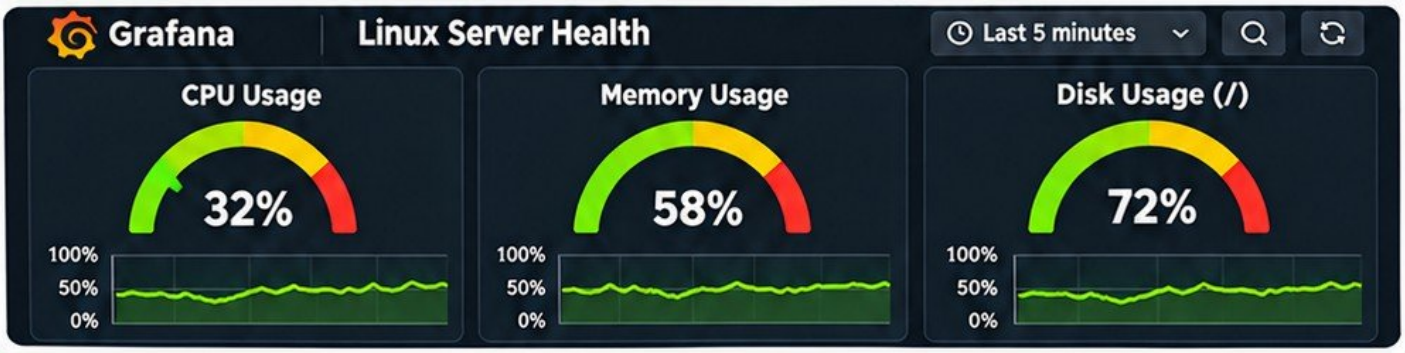
Memory Usage

```
100 * (1 - (node_memory_
MemAvailable_bytes /
node_memory_MemTotal_bytes))
```

Disk Usage

```
100 * (1 - (node_filesystem_
avail_bytes /
node_filesystem_size_bytes))
```

Example: Linux Server Health Dashboard



Practice

- Create a simple Linux server health dashboard.
- Include CPU, memory, and disk usage panels.
- Save and verify it updates in real time.

Visualization & Unit Tips

Metric	Recommended Visualization	Unit
CPU Usage	Gauge or Time series	%
Memory Usage	Gauge or Time series	% or GiB
Disk Usage	Gauge or Bar gauge	%

Make Grafana Dashboards Useful

Dashboard Time Ranges

Choose the right time range to analyze the data (e.g., 1h, 6h, 24h, 7d).

Refresh Intervals

Set how often the dashboard updates automatically (e.g., 5s, 30s, 1m, 5m).

Panel Titles and Descriptions

- Use clear, specific panel titles (e.g., CPU Usage).
- Add a short description to explain what the panel shows.

Thresholds and Units

- Add thresholds to highlight problem values (e.g., CPU > 80% in red).
- Use correct units (e.g., %, GB, MB, req/s).

Dashboard Variables

Use variables (e.g., $server) to select and switch between servers dynamically.

Legends and Query Labels

web-01
web-02
web-03

- Use legends to clearly identify what each line represents.
- Leverage query labels (e.g., instance, job) for meaningful names.

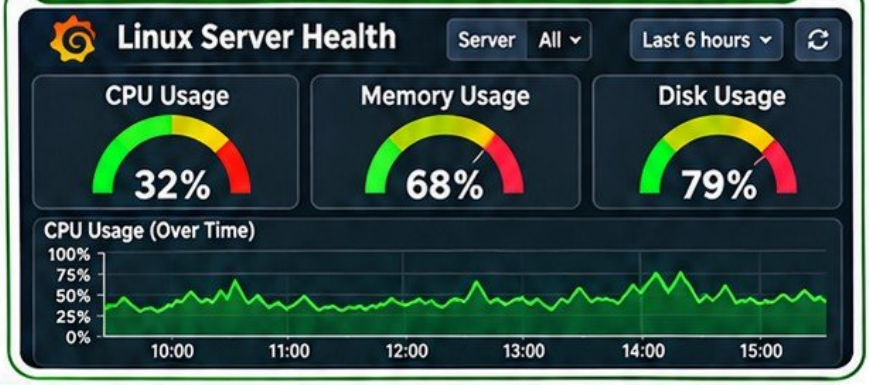
Avoid Misleading Graphs

- **Use the correct units** (e.g., % for utilization, bytes or GB for memory/disk).
- **Avoid wrong aggregations** (e.g., mixing different servers).
- **Choose the right visualization** (e.g., time series for trends, gauge for current value).

Practice

- [x] Improve your server dashboard for incident investigation.
- [x] Add clear titles, thresholds, variables, and useful units.
- [x] Verify the dashboard helps you quickly spot and understand issues.

Example: Improved Server Dashbcard



Introduction to Prometheus Alerting Rules

Metrics vs Alerts

Metrics: Measurable data (e.g., CPU usage, memory, request rate).

Alerts: Notifications triggered when a metric meets a defined condition (e.g., CPU > 80% for 5m).

Key Concepts

- **Alerting Rule:** A condition (expression) Prometheus evaluates regularly.
- **Alert Expression:** A PromQL query that defines when an alert should trigger.
- **for Duration:** How long the condition must remain true before the alert fires.
- **Alert States:** The lifecycle of an alert (inactive, pending, firing).

Alert States

Inactive	Condition is not met.
Pending	Condition is met but the for time has not elapsed.
Firing	Condition is met and the for duration has elapsed.

Example: Target-Down Alert

```yaml
groups:
 - name: server-alerts
   rules:
    - alert: TargetDown
      expr: up == 0
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "Target is down"
        description: "The target {{ $labels.instance }}
  has been down for more than 5 minutes."
```

Prometheus

How Prometheus Evaluates Rules

Evaluate expression (e.g., up == 0) → Check for duration (e.g., 5m) → Change state to Firing

Practice: Simulate a Failed Target

1. Stop the Node Exporter container (e.g., docker compose stop node-exporter).
2. Wait for the for duration (e.g., 5 minutes).
3. Check Prometheus → Alerts to see the TargetDown alert in Firing state.

Production Awareness

- Use a reasonable for duration (e.g., 5m or 10m).
- Add thresholds and stability checks to avoid alert noise from small, temporary spikes.

Common Mistake

Avoid alerts that fire on every small fluctuation. Use proper thresholds and a for duration to reduce false alarms.

Grafana Alerting and Notifications

What Is Grafana Alerting?

A built-in alerting system in Grafana that evaluates queries and sends notifications when a condition is met.

Key Concepts

- **Alert Rule:** A condition (query + threshold) that triggers an alert.
- **Evaluation:** How often Grafana checks the rule (e.g., every 1m).
- **Contact Point:** Where notifications are sent (e.g., email, Slack, webhook).
- **Notification Policy:** Rules that decide who gets notified and when.

Alert States

Normal	Condition is healthy.
Pending	Condition is met but still within the evaluation duration.
Firing	Condition has been true for the set duration and a notification is sent.

Setup: Configure a Test Notification

1. In Grafana, go to **Alerting → Contact points.**
2. Click "Add contact point" (e.g., Email, Slack, or Webhook).
3. Enter the destination details (e.g., test email or webhook URL).
4. Go to **Alerting → Alert rules** and create a simple rule (e.g., CPU > 80%).
5. Save and click "Test" to send a test notification.

Grafana Alerting vs Prometheus Alertmanager

Feature	Grafana Alerting	Prometheus Alertmanager
Where it runs	Inside Grafana	A separate component (Alertmanager)
Main purpose	Query, alert, and notify from Grafana	Receive alerts from Prometheus and route them (group, silence, retry)
Best for	Simple to medium use cases	Large, complex, and multi-team setups

Practice

- ☑ Create an alert rule in Grafana (e.g., CPU > 80% for 5m).
- ☑ Configure a contact point (e.g., test email).
- ☑ Trigger the alert (e.g., by simulating high CPU load) and confirm you receive a notification.

Security Reminder

- Protect notification credentials (e.g., email passwords, API tokens).
- Keep webhook URLs private (e.g., Slack, Teams, or custom APIs).
- Use secure storage (e.g., Grafana secrets or environment variables).

Troubleshoot Prometheus and Grafana Problems

Common Problems to Troubleshoot

- Prometheus targets showing DOWN.
- Missing metrics (expected metrics not found).
- Invalid PromQL expressions (query errors).
- Grafana data source connection errors.
- Dashboard panels showing "No data".

Symptoms and Likely Fixes

Symptom	Likely Fix
Target is DOWN	Check target URL, port, service status, and network.
Missing metrics	Verify exporter is running and /metrics is accessible.
PromQL error	Check metric name, labels, and function syntax.
Data source error	Confirm Prometheus URL (e.g., http://prometheus:9090).
"No data" on panel	Check time range, query, and that the metric actually exists.

Key Diagnostic Commands

```
# Check container status
docker compose ps
# View service logs
docker compose logs -f prometheus
# Check Prometheus targets
http://localhost:9090/targets
# Check exporter metrics
curl http://<exporter-ip>:9100/metrics
# Validate Prometheus config
promtool check config prometheus.yml
```

Configuration Reload vs Restart

Action	When to Use
Reload configuration	Use when you update prometheus.yml or alert rules (without stopping the service).
Restart service	Use when a reload fails, services are unhealthy, or you changed container settings (e.g., in docker-compose.yml).

Practice

- [x] Break a scrape target (e.g., stop Node Exporter).
- [x] Confirm it shows as DOWN in Prometheus.
- [x] Use logs and commands to find the cause.
- [x] Fix the issue (e.g., restart the exporter or correct the URL).
- [x] Verify the target returns to UP.

Troubleshooting Flow

1. Check the symptom (target, metric, query, or dashboard).
2. Verify service status and logs (docker compose logs).
3. Check configuration and connectivity (URL, port, network).
4. Fix the root cause (URL, config, query, or service).
5. Re-test and confirm (target UP, metrics visible, dashboard OK).

Monitor a Real Application

What Is Application Monitoring?

Observing the health, performance, and behavior of an application using metrics, logs, and alerts.

Key Application Metrics

- **HTTP Request Count:** Total number of requests received.
- **Request Rate:** Requests per second (RPS).
- **Error Rate:** Percentage of failed requests (e.g., 5xx errors).
- **Latency:** How long requests take to complete (e.g., 95th percentile).

The Four Golden Signals

Latency	How long a request takes (e.g., p95, p99).
Traffic	The amount of work (e.g., requests per second).
Errors	The rate of failed requests (e.g., 5xx, 4xx).
Saturation	How close the system is to its limits (e.g., CPU, memory, connections).

Instrument a Simple Python HTTP App

Python Flask App (with Metrics)

```python
from flask import Flask
from prometheus_client import Counter,
  Histogram, generate_latest

app = Flask(__name__)

REQUESTS = Counter(
  "http_requests_total", "Total requests",
  ["method", "status"])

LATENCY = Histogram(
  "http_request_duration_seconds",
  "Request latency in seconds")

@app.route("/")
def home():
  return "Hello, VERIQTA!"

@app.route("/metrics")
def metrics():
  rebeck generate_latest(), 200,
   {"Content-Type": "text/plain"}
```

Expose Metrics and Scrape With Prometheus

Prometheus Scrape Config (prometheus.yml)

```yaml
scrape_configs:
 - job_name: "python-app"
   static_configs:
   - targets: ["app:8000"]
  metrics_path: "/metrics"
```

Practice: Generate Requests and Observe

1. Run the Python app and expose /metrics.
2. Add the app to Prometheus scrape targets.
3. Generate traffic (e.g., curl or a simple loop).
4. Check metrics in Prometheus and Grafana.

How It Works (End-to-End Flow)

Python App (Exposes /metrics) → **Prometheus** (Scrapes Metrics) → **Grafana** (Visualizes & Alerts)

Monitor Application Health and Performance

Availability vs Performance

Availability: Whether the application is up and accessible (e.g., 99.9%).

Performance: How well it responds (e.g., speed, latency, and throughput).

Key Metrics to Monitor

Request Rate: Number of requests per second (RPS).

HTTP Error Ratio: Percentage of failed requests (e.g., 5xx or 4xx).

Latency: Response time, measured using percentiles (e.g., p50, p95, p99).

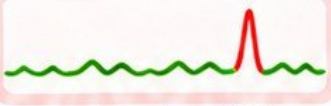
Latency Percentiles (Histogram)

Use histogram metrics to calculate latency percentiles.

```
# 95th percentile latency
 histogram_quantile(0.95,
  sum by (le) (rate(
 http_request_duration_seconds_bucket[5m]))
```

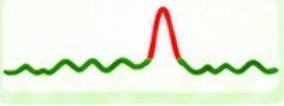
Why Averages Can Be Misleading

Average: 200 ms



Looks fine, but hides occasional slow requests (e.g., 5s).

p95: 4.8 s



Shows the real user experience for slower requests.

Build Health Panels in Grafana



- Request rate (RPS)
- HTTP error ratio (%)
- Latency (p50, p95, p99)
- Application availability (up / down)

Service Level Indicators (SLIs)

SLIs are measurable indicators that show the health and performance of your service, such as:

Availability	Latency	Error Rate
(e.g., 99.9%)	(e.g., p95 < 500 ms)	(e.g., < 1%)

Practice: Investigate HTTP Errors

1. Simulate an increase in HTTP errors (e.g., return 500 for some requests).
2. Observe the error rate panel in Grafana.
3. Check the request rate and latency to see the impact.
4. Identify the cause (e.g., code error, dependency failure), fix it, and confirm the metrics recover.

Example: Key PromQL Queries

Purpose	PromQL Query
Request rate (RPS)	`sum(rate(http_requests_total[5m]))`
HTTP error ratio	`sum(rate(http_requests_total {status=~"5.."}[5m])) / sum(rate (http_requests_total[5m]))`
p95 latency	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_ bucket[5m])))`

Prometheus and Grafana in Production

Monitoring Reliability

Ensure your monitoring stack is stable, accurate, and available when you need it (e.g., during incidents).

Prometheus Storage and Retention

Prometheus Storage

- Uses a local time-series database.
- Stores metrics on disk (e.g., /var/lib/prometheus).
- Control how long data is kept with retention (e.g., --storage.tsdb.retention.time=15d).

Scrape Frequency vs Storage Growth

- **Scrape frequency:** How often Prometheus collects metrics (e.g., 15s, 30s, 1m).
- Lower frequency = less data and smaller storage.
- Higher frequency = more detail but faster storage growth.

Label Cardinality

The number of unique label combinations (time series).

- Keep labels low and meaningful (e.g., job, instance, env).
- Avoid high-cardinality labels (e.g., user_id, request_id, session_id).

Authentication, Access Control, and TLS

- Secure access to Prometheus and Grafana (e.g., username/password, OAuth, or SSO).
- Use role-based access control (RBAC) in Grafana.
- Enable TLS (HTTPS) to encrypt traffic (e.g., with a reverse proxy like Nginx or Traefik).

Backups and Persistent Storage

- Use persistent storage (e.g., Docker volume, EBS, or a PV in Kubernetes).
- Back up critical data (e.g., Prometheus data and Grafana configuration/dashboards).
- Prevent data loss during upgrades, crashes, or system migration.

Monitor Kubernetes Workloads

- Monitor pods, deployments, services, and nodes.
- Use kube-state-metrics and node-exporter.
- Visualize cluster and workload health in Grafana (e.g., CPU, memory, pod restarts).

When to Use Remote Storage or HA

- Use remote storage (e.g., Thanos, Cortex, Mimir, or VictoriaMetrics) for long-term data and large scale.
- Use a highly available (HA) setup when you need high uptime (e.g., multiple Prometheus replicas + shared storage).
- Consider this for multi-cluster, large teams, or production-critical systems.

Production Checklist

- Set retention and storage limits.
- Tune scrape frequency.
- Control label cardinality.
- Secure access (RBAC + TLS).
- Use persistent storage and backups.
- Plan for scale (Kubernetes, remote storage, or HA).

Build a Complete Monitoring Stack

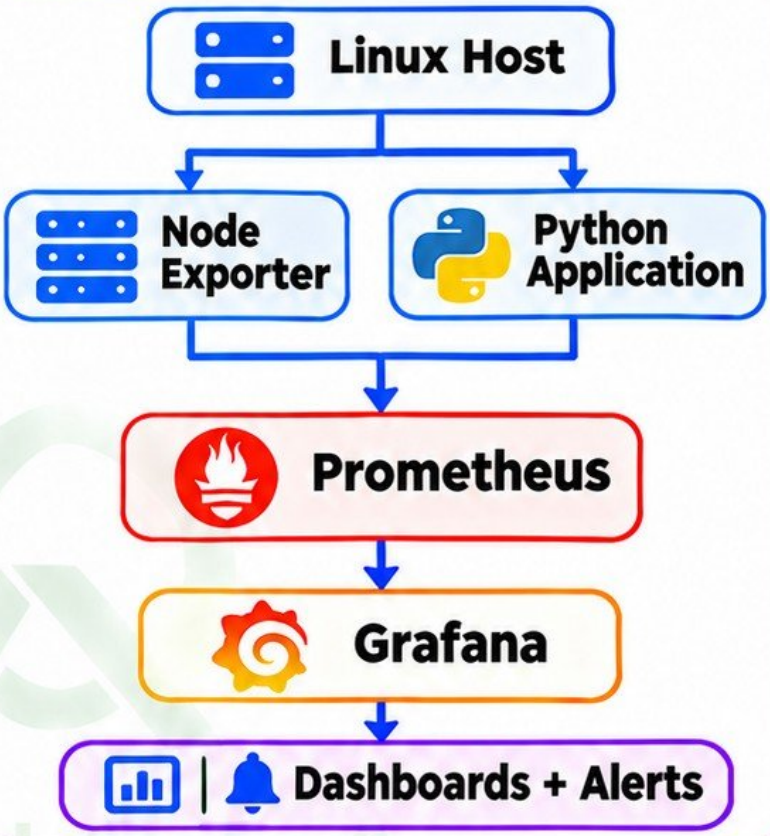
Project

Monitor a Linux Server and Python Application

Objectives

1. Create a Docker Compose monitoring project.
2. Deploy Prometheus and Grafana.
3. Deploy Node Exporter.
4. Deploy an instrumented Python application.
5. Configure all Prometheus scrape targets.
6. Create a server health dashboard.
7. Create an application performance dashboard.
8. Configure one meaningful alert.
9. Verify metrics collection and dashboard accuracy.

Architecture



Monitoring Stack Components

Prometheus	Collects and stores metrics (time series).
Grafana	Visualizes metrics and manages alerts.
Node Exporter	Provides Linux server metrics (CPU, memory, disk, etc.).
Python App	Application that exposes metrics at /metrics.

Key Dashboards

Server Health Dashboard
CPU, memory, disk, load, and uptime.

Application Performance Dashboard
Request rate, error rate, latency (p50/p95), and availability.

Example Alert (Meaningful)

High HTTP Error Rate
Alert when HTTP 5xx errors are > 5% for 5 minutes.

Verification Checklist

- [x] Prometheus targets are UP.
- [x] Metrics are visible in Grafana.
- [x] Dashboards show correct and changing data.

Production Incident Simulation and Final Assessment

Incident Scenario



Your application becomes **slow** and starts returning **errors**.

What You Will Do

Observe and Analyze (Grafana) → Query and Investigate (PromQL) → Fix, Verify, and Document (Resolution)

Investigation Steps

1. Observe symptoms in **Grafana**.
2. Check **CPU** and **memory** usage.
3. Examine application **request rate**.
4. Investigate **HTTP error** ratios.
5. Inspect request **latency**.
6. Use **PromQL** to identify abnormal behavior.
7. Correlate metrics with **application logs**.
8. Identify the likely **cause** using evidence.
9. Apply a safe **corrective action**.
10. **Verify recovery** using metrics.

Key Metrics to Check

Metric	What It Tells You
CPU Usage	Is the server/app under high load?
Memory Usage	Is the app running out of memory?
Request Rate (RPS)	Is traffic higher than normal?
HTTP Error Ratio	Are errors increasing (e.g., 5xx, 4xx)?
Latency (p50/p95)	Are responses slower than expected?

Useful PromQL Examples

```
# Request rate (RPS)
sum(rate(http_requests_total[1m]))
# HTTP error ratio
sum(rate(http_requests_total{status=~"5.."}[1m])) /
sum(rate(http_requests_total[1m]))
# Latency (p95)
histogram_quantile(0.95,
 sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Resolution Flow

Detect (Metrics) → Diagnose (Cause) → Fix (Safe Action) → Verify (Recovery) → Document (RCA + Prevention)

Incident Documentation



- What happened (summary).
- Root cause (with evidence).
- What was fixed (action taken).
- Preventive actions (to avoid repeat).

Final Assessment



Demonstrate that you can:

- Install and configure Prometheus and Grafana.
- Collect and query metrics with PromQL.
- Build useful dashboards.
- Troubleshoot a real performance issue.
- Use alerting effectively.
- Document the incident, root cause, and preventive actions.